Project Report

VIT BHOPAL UNIVERSITY

Sehore, Madhyapradesh

Submitted to : Dr. j Manikandan

Submitted by : Aayush kumar

Registration no : 25BAI10975

Couse Title : Fundamental of AI

Course code : CSA2001

Program : B.tech

Session : 2025-26

Branch : Scai

Date : 28th march 2026

1. Introduction

In today's world, we are constantly told that "the internet never forgets." Every text, post, and email usually leaves a digital trail that lasts for years. **SecureVault** was born out of a simple question: *What if we could make a conversation that truly disappears?* This project is a Python-based app designed to show off two big ideas in security: **Symmetric Cryptography** (using one "lock" for both locking and unlocking) and **Volatile Key Management** (keys that vanish when you're done). The goal isn't just to scramble text; it's to create a "safe zone" where your secrets only exist for the few minutes you're actually talking. Once the session is over, the data is gone for good.

2. Project Objective

The heart of SecureVault is the idea of a **"zero-footprint"** messenger. Most security tools have a "key problem." They create a secret key and save it as a file on your computer (like a .key file). But if a hacker gets into your system, that file is the first thing they'll look for.

SecureVault takes a different path. It generates a unique **Master Key** that lives strictly in your computer's **RAM** (its "short-term memory"). Because RAM clears itself every time a program closes or a computer restarts, the key is never actually written onto your hard drive. This means that even if someone stole your laptop five minutes after you finished a chat, they would find absolutely nothing to unlock. The key is mathematically destroyed.

3. Technical Architecture & Tools

I chose **Python 3.x** to build this because it's reliable, easy to read, and has some of the best security tools available for developers today.

The main tools in my kit include:

- **Cryptography (Fernet):** This is the "heavy lifter" of the project. I used the **Fernet** system, which is a gold standard for security. It uses **AES** (the same encryption banks use) to scramble the message, and **HMAC** to make sure no one

"tampered" with the code while it was being sent. It's like a digital wax seal on an envelope—if the seal is broken, you know the message isn't safe.

- **Object-Oriented Programming (OOP):** I wrapped the whole project inside a SecureMessenger class. This makes the code "modular," meaning it’s organized like a LEGO set. You could easily take this code and plug it into a bigger website or a mobile app without starting from scratch.
- **OS Module:** I used this to handle the "housekeeping." Specifically, it clears the terminal screen after every message. This prevents "shoulder surfing"—those awkward moments where someone walking by your desk could read your private chat history just by looking at your screen.
- **Time Module:** I added tiny delays to the program to simulate a "secure tunnel." This makes the app feel more interactive and gives the user a second to see the encryption process happening in real-time.

4. Implementation Details

The app follows a very specific "life cycle" to make sure no data leaks out:

- **Initialization:** The second you hit "Run," the app creates a one-of-a-kind session key. This key is random and unique; no two people using the app will ever have the exact same key.
- **The Encryption Loop:** When you type a secret, the app turns your words into bytes (the 1s and 0s computers love) and then scrambles them. The result is a **Base64 "token"**—a long string of random-looking gibberish.
- **Simulation & Decryption:** To show how the tech works, the app displays a "Stack Trace" (the gibberish) to show what a hacker would see. Then, it immediately uses that hidden RAM-based key to turn that gibberish back into your original words so you can see the final result.

5. Security Features & Testing

I built SecureVault with a "safety first" mindset, including several ways to prevent accidents:

- **Key Volatility:** This is the star of the show. By not saving the key to the disk, we remove the biggest risk in security: a stolen key file.
- **Error Handling:** Cryptography can be "picky." If a single letter of the encrypted code is missing, most programs would just crash. I wrote the code to catch these errors gracefully, telling the user there was a problem rather than just breaking.
- **Graceful Termination:** We've all accidentally closed a window or hit the wrong buttons. I included a "panic catch" (KeyboardInterrupt) so that if you hit Ctrl + C, the program shuts down politely and wipes the memory session instantly.

6. Conclusion

SecureVault proves that you don't need a massive server or expensive hardware to have top-tier privacy. By using **RAM-only storage** and strong **AES encryption**, I've created a working model for safe, temporary communication.

In the future, I'd love to take this even further. Imagine connecting two different computers over the internet using a **socket-based network** or using a **Diffie-Hellman handshake**—a clever way for two people to agree on a secret key without ever meeting in person. For now, SecureVault is a solid step toward a more private digital world.